Market and Reference Data: A Specification for the Ledger Platform

MVP, v2.0

Data Engineering Specification

May 2026

The document covers three things. First, the catalogue of data the platform needs — what each kind of record looks like, what fields it carries, where the data comes from, and how each concept maps to CDM. Second, provenance: every record records its source, when the underlying event happened, and when our system received it. Third, bitemporal storage: nothing is silently overwritten, and a question about what we knew yesterday gets a different answer from a question about what is true today. A fourth section is the storage shape the pricing library needs to load this data quickly.

The catalogue comes first because it is what we will be writing schemas against on Monday morning. The provenance, bitemporal, and fast-loading disciplines apply to every entry in the catalogue, and are stated once each.

1 The platform in one paragraph

The Ledger records portfolio activity as moves of *units* (currencies, shares, bonds, OTC contracts, on-chain obligations) between *wallets*. Every move conserves quantity per unit: what leaves one wallet enters another. The platform itself is not a pricing engine; portfolio value is computed by combining wallet balances with prices and curves drawn from the data substrate this document specifies. The data engineering team owns that substrate and the boundary at which outside data enters it.

2 The catalogue

This section specifies the kinds of record the substrate stores. For each concept the document gives a definition, the fields, where the data comes from, why we need it, and a worked example with values populated.

The provenance fields described in Section 3 (`source`, t_{obs}, t_{known}, `restates_ref`) are carried on every record in every concept below. They are not repeated in each field list; the worked examples in this section likewise omit them to keep the field tuples readable, and they should be treated as implicit on every record per the convention stated here. The single bitemporal worked example in Section 4 carries them explicitly because the example is about them. Where a concept declares a `status` field, it is a per-concept closed list; the same field name on different concepts (currencies, listed equities, raw observations, . . .) carries different values, not a global enum.

2.1 Currencies

A currency is a unit of money. The ISO 4217 standard maintains the canonical list of currencies and their codes. Fields: `iso_code` (three letters), `name`, `minor_units` (integer; e.g. 2 for USD, 0 for JPY), `status` (`active`, `historic`). Carrier: ISO 4217.

We need this because every monetary amount in the system is a number plus a currency code, and code-to-name lookups, FX conversions, and rounding all depend on a single agreed table of currencies and their minor-unit precision. Without it, a JPY amount rounded to two decimals becomes a 100× error.

CDM mapping. CDM does not carry a separate currency-master table; the currency code itself is the model citizen. `ISOCurrencyCodeEnum` in `base-staticdata-asset-common-enum.rosetta` enumerates the ISO 4217 codes, and `CurrencyCodeEnum` (same file) extends it with the FpML non-ISO scheme for offshore and historical codes. The companion type `Cash extends AssetBase` (`base-staticdata-asset-common-type.rosetta`) is how a currency is held as an asset: the currency code lives in the inherited `AssetIdentifier` with `AssetIdTypeEnum -> CurrencyCode`. The display-name and `minor_units` attributes are absent from CDM; the Ledger keeps them in a local currency-master table, joined to CDM by code.

Example. `(USD, "US Dollar", 2, active)`; `(JPY, "Japanese Yen", 0, active)`.

2.2 Calendars and date conventions

Two related shapes. A *business-centre calendar* is the holiday list for a city or market: `centre_code` (e.g. `USNY`, `GBLO`, `EUTA`, `JPTO`), `holiday_dates` (list of dates), `valid_from`, `valid_to`. Carrier: a vendor calendar service (e.g. Copp Clark) or the FpML business-centre service. *Date conventions* are two closed lists: a day-count fraction (`ACT/360`, `ACT/365.FIXED`, `30/360`, `30E/360`, `ACT/ACT.ISDA`) and a business-day convention for date adjustment (`Following`, `ModifiedFollowing`, `Preceding`, `ModifiedPreceding`, `NONE`).

We need this because every coupon date, payment date, fixing date, and reset date is a calendar-adjusted date evaluated against a specific business centre under a specific convention. Two systems running the same trade against different calendars compute different cashflows on different days. A single trade leg can carry distinct business centres for different purposes — payment-date adjustment, fixing-date adjustment, and rate-publication — so a leg's `business_centres` field is per-purpose (payment, fixing, reset), not one combined list.

CDM mapping. The business-centre side is well-supported. `BusinessCenters` in `base-datetime-type.rosetta` carries `businessCenter : BusinessCenter (0..*)` — a list of `BusinessCenter` wrappers, each holding a `BusinessCenterEnum` from `base-datetime-enum.rosetta` (the standard FpML codes `USNY`, `GBLO`, `EUTA`, `JPTO`, etc.). The per-purpose split is honoured by CDM: a payout's `calculationPeriodDates`, `paymentDates`, and `resetDates` each carry their own `businessDayAdjustments` sub-record with its own `businessCenters`, which is precisely the payment-versus-fixing-versus-reset distinction we need. Day-count and business-day conventions are CDM enums: `DayCountFractionEnum` (in `base-datetime-daycount-enum.rosetta`) and `BusinessDayConventionEnum` (in `base-datetime-enum.rosetta`). The dated list of holidays itself is not modelled by CDM as a stored type — CDM names a centre, but the resolution of that centre to a concrete date list is delegated to the implementation. The Ledger holds the resolution table locally, keyed by `BusinessCenterEnum`.

Example. A USD floating leg paying quarterly under USNY calendar, day-count `ACT/360`, business-day convention `ModifiedFollowing`. The 2026 USNY calendar lists 2026-01-01, 2026-01-19, 2026-02-16, ..., 2026-12-25 as holidays.

The shape of `holiday_dates` — an array column on the calendar row or a child `calendar_holiday` table keyed by `(centre_code, date)` — is left to the warehouse design; the catalogue requires only that the set of holiday dates is retrievable per `(centre_code, valid_from, valid_to)` as a single answer.

2.3 Parties

A party is a legal entity transacted with: a counterparty, a custodian, an exchange, an issuer, an internal entity. Fields: `lei` (20-character ISO 17442 identifier), `legal_name`, `jurisdiction` (ISO 3166 country code), `lei_status` (`Issued`, `Lapsed`, `Retired`, `Annulled`, `Duplicate`, with `PendingValidation`, `PendingTransfer`, `Merged` and any further values published by GLEIF admitted as the registry evolves), `bic` (optional), `mic` (optional, only when the party is an exchange or trading venue). Carrier: GLEIF for the LEI record itself; vendor or internal master for cross-references.

We need this because every contract, holding, and obligation has at least one named legal party on it, and reconciling a counterparty across vendors is hopeless without a single legal-entity identifier. A lapsed LEI is not the same as an active one; status changes are recorded as new rows, not by mutating the existing record.

CDM mapping. `Party` (`base-staticdata-party-type.rosetta`) carries `partyId : PartyIdentifier (1..*)` for one or more identifiers, `name : string (0..1)`, plus optional `businessUnit`, `person`, `personRole`, and `contactInformation`. `PartyIdentifier` holds the identifier string plus an `identifierType : PartyIdentifierTypeEnum (0..1)` tagging the identifier kind (e.g. `LEI`); the FpML party-id scheme rides on the field's `[metadata scheme]` annotation. `lei_status` is not on CDM `Party`; the Ledger carries it as a sidecar attribute joined by LEI. `jurisdiction` has no CDM home — `LegalEntity` (same file) carries only `name` and `entityIdentifier`, no jurisdiction field — so the Ledger stores it as a local field (ISO 3166 country code). `bic` and `mic` are additional `PartyIdentifier` entries with their own `identifierType` values.

Example. `(W22LROWP2IHZNBB6K528, "Goldman Sachs International", GB, Issued)`.

2.4 Listed equities

A listed equity is a share traded on at least one regulated exchange. Fields: `ticker`, `isin`, `cusip` (US listings), `sedol` (UK listings), `figi`, `mic` (primary listing exchange code), `currency`, `sector_code_system` (`GICS` or `ICB`), `sector_code` (the value expressed in that system), `board_lot` (integer), `status` (`active`, `delisted`, `suspended`). Carrier: a market-data vendor (e.g. Bloomberg, Refinitiv) cross-checked against the exchange's own master.

We need the multi-identifier mapping because a single share trades under different names in different systems: ticker on the trading screen, ISIN on a settlement instruction, FIGI in a vendor feed, CUSIP on a US custody report. A trade reported under one identifier must reconcile to a position reported under another.

CDM mapping. CDM models a listed equity as `Security extends InstrumentBase` (`base-staticdata-asset-common-type.rosetta`) with `securityType = SecurityTypeEnum.Equity` and an optional `equityType : EquityType (0..1)`. The inherited `InstrumentBase` carries the identifier list (via `Asset`'s `AssetIdentifier` with `AssetIdTypeEnum` values such as `Isin`, `Cusip`, `Sedol`, `Figi`, `Ticker`). `Security` is a leaf in the top-level `Instrument` choice (same file), which is itself a branch of the `Asset` choice. The `mic` for the primary listing is an additional identifier on the same record. Missing in CDM: `sector_code_system` and `sector_code` (no GICS/ICB classification type in CDM 6.0.0). The Ledger carries these as a local `IndustryClassification` sidecar joined to the CDM record by instrument identifier; an extension would add a `sectorClassification : SectorClassification (0..1)` attribute to `EquityType`.

Example. Apple Inc.: `(AAPL, US0378331005, 037833100, --, BBG000B9XRY4, XNAS, USD, GICS, 4520, 1, active)`.

2.5 Listed derivatives

A listed derivative is a future or option traded on a regulated exchange. Fields: `root` (e.g. `ES` for E-mini S&P), `exchange_mic`, `contract_month` (`YYYY-MM`), `instrument_type` (`future` or `option`), `strike` (options only), `option_type` (`call` or `put`, options only), `multiplier`, `tick_size`, `settlement_type` (`cash` or `physical`), `underlier_ref` (foreign key to the underlying instrument), `currency`, `status`. Carrier: the exchange's own product specification, mirrored by vendors.

We need the multiplier and tick size because they are the bridge between price and cash: a 0.25 tick on an E-mini S&P at multiplier 50 is $12.50 per contract per tick, and a system that loses the multiplier prices every contract at 1/50 of its real value.

CDM mapping. `ListedDerivative extends InstrumentBase` (`base-staticdata-asset-common-type.rosetta`) is the CDM type. It carries `deliveryTerm : string (0..1)` (the contract-month code, e.g. `Z23`), `optionType : PutCallEnum (0..1)` (absent for futures, set for options), and `strike : number (0..1)` (set only for options, enforced by the type's `Options` condition). The `root`, `exchange_mic`, and other identifiers ride on the inherited `AssetIdentifier` list. Missing in CDM at the type level: `multiplier`, `tick_size`, `settlement_type`, `underlier_ref`. CDM resolves these by reference to the listed-product master at the exchange; the Ledger keeps them locally on the listed-derivative record because we need them at pricing time without an extra round-trip. An extension would add a `contractSpecification : ListedContractSpec (0..1)` attribute to `ListedDerivative`.

Example. The June 2026 E-mini S&P 500 future: `(ES, XCME, 2026-06, future, --, --, 50, 0.25, cash, SPX, USD, active)`.

2.6 Bonds

A bond is a fixed-income instrument with a defined maturity, coupon stream, and issuer. Fields: `isin`, `issuer_lei`, `coupon_rate` (decimal, or the literal `FLOAT` for floating-rate notes), `coupon_frequency` (`annual`, `semi`, `quarterly`, `monthly`, `zero`), `maturity_date`, `issue_date`, `accrual_start_date` (the date from which interest accrues; usually equal to `issue_date` but distinct for bonds with a delayed first coupon), `first_coupon_date` (the date of the first scheduled coupon payment; distinct for odd-first-coupon bonds), `face_value`, `currency`, `day_count`, `bd_convention`, `seniority` (`senior`, `subordinated`, `secured`), `callable` (boolean), `call_schedule` (optional list of call dates and prices). Carrier: vendor bond master cross-checked against issuer's prospectus.

We need the day-count, frequency, and convention on the bond record because accrued interest and clean-vs-dirty price calculations depend on them, and a bond stripped of these fields cannot be priced or settled correctly. Without `accrual_start_date` and `first_coupon_date`, an odd-first-coupon bond cannot be reproduced from the field list alone. Note that whether the schedule itself is calendar-adjusted is asset-class-dependent: US Treasury coupon dates are unadjusted by market convention (`bd_convention=NONE`); standard corporate bonds typically adjust under `Following` or `ModifiedFollowing`. The pairing of `ACT/ACT.ISDA` with `NONE` on a UST is intentional, not an oversight.

CDM mapping. A bond is a `Security extends InstrumentBase` (`base-staticdata-asset-common-type.rosetta`) with `securityType = SecurityTypeEnum.Debt` and `debtType : DebtType (0..1)`. `DebtType` holds a list of `DebtEconomics (0..*)` with `seniority : DebtSeniorityEnum`, `interest : DebtInterestEnum` (fixed/floating/zero), `principal : DebtPrincipalEnum`, `secured : SecuredDebt (0..1)`, and `redemption : DebtRedemption (0..1)` (callable schedule rides here). The ISIN, issuer LEI, and other identifiers are `AssetIdentifier` entries on the inherited `Asset`. The full coupon-schedule arithmetic — `coupon_rate`, `coupon_frequency`, `maturity_date`, `issue_date`, `accrual_start_date`, `first_coupon_date`, `day_count`, `bd_convention` — is not held as scalars on `Security`; CDM expresses these as a payout (`InterestRatePayout` in

product-asset-type.rosetta, or FixedPricePayout in product-template-type.rosetta) attached to the bond when it appears in a contract. For static bond-master storage the Ledger flattens these onto the bond row so that pricing does not need to resolve a payout structure for every reference bond.

Example. A 5-year US Treasury note: (US91282CJZ59, 254900HROIFWPRGM1V77, 0.04250, semi, 2031-04-30, 2026-04-30, 2026-04-30, 2026-10-31, 1000, USD, ACT/ACT.ISDA, NONE, senior, false, --).

2.7 OTC products

An OTC product is a contract whose terms are negotiated bilaterally rather than listed on an exchange. The substrate stores the *terms*, not a vendor product identifier. The shape varies by product type. The common header is product_type (closed list for the MVP: IRS, FXForward, FXSwap, EquityOption, CDS; new product types are admitted by adding an entry to the dictionary of supported types and the corresponding payoff-parameters shape, not by silent expansion of the enum), effective_date, termination_date, plus a payoff-parameters block whose contents are determined by the product type. For an interest rate swap the payoff block contains notional, currency, a fixed leg (rate, frequency, day_count, business_centres), and a floating leg (index_name, tenor, spread, frequency, day_count, business_centres, reset_offset, compounding where applicable). Carrier: internal trade capture. Industry identifiers — the OTC ISIN and the UPI (ISO 4914) — are recorded as additional identifiers on the terms record, not as substitutes for the terms. On-chain obligations (smart-contract derivatives) use the same terms shape, with the on-chain contract address recorded as one further identifier field.

We need the full economic terms, not a vendor identifier, because the platform is the source of truth for the contract; if a vendor reissues identifiers or goes away, the contract still has to be valued and settled.

CDM mapping. CDM is most opinionated here. An OTC product is a NonTransferableProduct (product-template-type.rosetta) whose core is economicTerms : EconomicTerms (1..1). EconomicTerms holds effectiveDate and terminationDate, plus a payout : Payout choice with exactly eight branches: AssetPayout, CommodityPayout, CreditDefaultPayout, FixedPricePayout, InterestRatePayout, OptionPayout, PerformancePayout, SettlementPayout (forward settlement is carried by SettlementPayout, not by a separate ForwardPayout). The choice declaration is in product-template-type.rosetta; the interest-rate, credit-default, and commodity leaf-type bodies live in product-asset-type.rosetta. For an IRS, the swap is a pair of InterestRatePayout entries (one fixed, one floating); the floating leg references a FloatingRateIndex (observable-asset-type.rosetta, a branch of the Index choice) whose floatingRateIndex attribute is a FloatingRateIndexEnum value from base-staticdata-asset-rates-enum.rosetta (the live filename — the v1 reference list named this as floatingrateindex-enum.rosetta, which does not exist on the current master branch). The compounding flag rides on the chosen enum value; SOFR's overnight-compounded variant is USD_SOFR_COMPOUND. UPI (ISO 4914) and ISIN-for-OTC are added as further ProductIdentifier entries (with ProductIdTypeEnum values) on the surrounding NonTransferableProduct. On-chain obligations have no first-class CDM type today; the Ledger carries the contract address as an additional ProductIdentifier with a local source value.

Example. A 5-year USD SOFR OIS, traded 2026-04-30: product_type=IRS, effective_date=2026-05-04, termination_date=2031-05-04, notional=10,000,000, currency=USD, fixed_leg=(rate=0.04150, frequency=annual, day_count=ACT/360, business_centres={payment:[USNY]}), float_leg=(index_name=USD-SOFR, tenor=1D, spread=0.0, frequency=annual, day_count=ACT/360, business_centres={payment:[USNY], fixing:[USNY], reset:[USNY]}, reset_offset=-2BD, compounding=COMPOUND-IN-ARREARS).

2.8 Observable definitions

An observable definition is the *key* under which raw market values are recorded; it does not carry any value itself. Separating the definition from the values lets us record the same observable across many timestamps and many sources without repeating the identifying tuple. Fields depend on what is being observed. For an equity quote: `kind=equity_quote`, `ticker`, `mic`, `quote_type` (`bid`, `ask`, `mid`, `last`, `close`), `currency`. For an FX print: `kind=fx_print`, `pair` (e.g. `EURUSD`), `quote_type`, plus two optional fields used by non-deliverable forward fixings: `settlement_currency` (the currency in which the NDF cashflow is settled, e.g. USD for USDKRW NDF) and `fixing_source` (the named fixing convention, e.g. `KFTC18` for KRW). For an interest-rate fixing: `kind=ir_fixing`, `index_name`, `tenor`.

We need explicit definitions because raw observation tables grow into the billions of rows; without a stable, normalised key, joins are unmaintainable and definitional changes (a new quote type, a new fixing source) would force schema migrations on the values table.

CDM mapping. CDM does not separate "definition" from "value" on an observation; it pushes the identifying tuple inside `ObservationIdentifier` (`observable-event-type.rosetta`) and combines it with the value in a single `Observation` row (rootType, same file). `ObservationIdentifier` holds `observable : Observable (1..1)`, `observationDate : date (1..1)`, `observationTime : TimeZone (0..1)`, `informationSource : InformationSource (0..1)`, and `determinationMethodology : DeterminationMethodology (0..1)`. `Observable` is a three-branch choice (`Asset | Basket | Index`) in `observable-asset-type.rosetta`; `Curve` is a separate, deprecated, top-level type in the same file — not a branch of `Observable` — which is why calibrated curves have no CDM-native home. For an equity quote the `Observable` resolves to a `Security` via the `Asset` branch; for an FX print it resolves to `ForeignExchangeRateIndex` (with the optional NDF `fixing_source` riding directly on `ObservationIdentifier.informationSource` — KFTC18 is exactly an `InformationSource` on the wire); for an IR fixing it resolves to `FloatingRateIndex` with `indexTenor : Period (0..1)`. The Ledger's definition-versus-value split is local: we store the `ObservationIdentifier` tuple once as a definition row keyed by surrogate id and join the value rows to it, because billions of fixings sharing the same identifying tuple do not need that tuple repeated.

Example. `(equity_quote, AAPL, XNAS, close, USD)`; `(fx_print, EURUSD, mid)`; `(ir_fixing, USD-SOFR, 1D)`.

2.9 Raw market observations

A raw market observation is a single value delivered against an observable definition. Fields: `definition_ref` (foreign key into observable definitions), `value` (numeric), `value_time` (the printed time of the quote or trade as reported by the source), `status` from the closed list {`consensus`, `single_source`, `quarantined`}. When the same observable is delivered by two or more sources that agree within tolerance the record is marked `consensus`; a unique authoritative source carries `single_source`; observations failing admission carry `quarantined` and are excluded from downstream computations. Carrier: market-data vendors, exchanges, and calculation agents.

`value_time` is the source-reported instant of the underlying market event (the printed quote or trade time on the vendor wire); the platform-canonical observation time t_{obs} described in Section 3 is the same instant normalised to UTC and carried on every record. For most ingests they coincide; they diverge when the source uses a non-UTC clock or restates a printed time and we preserve both for traceability.

We need the explicit `status` field because a downstream pricing or risk calculation must be able to filter out bad observations with a SQL predicate on a typed column, not by guessing which records were meant to be excluded. The consensus tolerance — how close two source values must be to mark the record `consensus` — is a per-observable-kind parameter held in a configuration reference table (a fixing tolerance for an IR index is not the same as a price tolerance for a thinly-traded

share). Status is assigned at admission by the admission service named in the record's `source` field; a status change is itself a restatement under the bitemporal rule in Section 4. When two named sources deliver the same observable and disagree beyond the tolerance, both source rows are retained and both land as `quarantined`; the full multi-source reconciliation policy — which source wins, manual override workflow, alerting — is a follow-on artifact, not part of this document.

CDM mapping. `Observation` (`observable-event-type.rosetta`, marked `[rootType]`) carries `observedValue : Price (1..1)` and `observationIdentifier : ObservationIdentifier (1..1)`. The numeric value rides in `Price extends PriceSchedule` (`observable-asset-type.rosetta`), which itself extends `MeasureSchedule` with a unit and currency. `Observation` is annotated `[metadata key]`, so other CDM records can refer to a stored observation by hash. CDM does not name a `status` field, a `value_time` field distinct from the observation's logical date, or a `source` attribute on the observation itself; the source is expected to ride at the surrounding `Event` or `TradeState` level. The Ledger carries `status`, `value_time`, and the provenance fields locally on the observation row because admission, quarantine, and bitemporal restatement all happen at observation granularity for us, not at trade granularity.

Example. `(definition=(equity_quote, AAPL, XNAS, close, USD), value=189.32, value_time=2026-04-30 16:00 ET, status=consensus, source="Bloomberg BPIPE")`.

2.10 Calibrated market objects

A calibrated market object is a curve or surface fitted from raw observations: a yield curve, a volatility surface, an FX surface, a hazard curve. Fields: `kind` (`yield_curve`, `vol_surface`, `fx_surface`, `hazard_curve`), `valuation_date`, `cut_time` (the wall-clock time of the inputs actually used — end-of-day for daily marks, but intraday for intraday calibrations), `currency_or_underlier`, `calibration_method`, `method_version`, `knot_points` (a list of `(tenor, value)` for curves; a list of `(strike, expiry, value)` for surfaces), `input_observation_refs` (foreign keys back into raw observations). Carrier: the platform's calibration process; the inputs come from raw observations.

We need the `input_observation_refs` because reproducing yesterday's curve is the only way to reproduce yesterday's prices, and that needs both the calibration code version and the exact set of raw observations consumed. The refs are frozen at the moment of calibration: each ref points at a specific bitemporal row of the raw-observations table, identified by its surrogate id, and resolves to that row even if a later restatement supersedes it. A restated input does not implicitly restate the curve; restating the curve is a separate event that re-runs the calibration against the new inputs and writes a new bitemporal row.

CDM mapping. Missing in CDM 6.0.0. CDM has `Curve` and `InterestRateCurve` stub types in `observable-asset-type.rosetta`, both annotated `[deprecated]`, with no knot-point representation, no surface type, and no calibration-method or input-refs attribute. The Ledger keeps a fully local shape: a `CalibratedMarketObject` row with the fields above and a foreign-key list back into the raw-observations table. An extension would add a `CalibratedCurve`/`CalibratedSurface` type alongside CDM's `Curve` stub, with a `knotPoint : KnotPoint (1..*)` list and an `inputObservation : Observation (0..*) [metadata reference]` list pointing back at the consumed observations.

Example. A USD yield curve as of 2026-04-30 EOD: `kind=yield_curve, valuation_date=2026-04-30, cut_time=2026-04-30 17:00 ET, currency_or_underlier=USD, calibration_method=bootstrap, method_version=2.3.1, knot_points=[(1M, 0.0483), (3M, 0.0479), (6M, 0.0471), (1Y, 0.0455), (2Y, 0.0421), (5Y, 0.0410), (10Y, 0.0418), (30Y, 0.0430)]`.

2.11 Lifecycle events

A lifecycle event records something that happens to an instrument or a trade after it begins to exist: corporate actions (dividends, splits, mergers, tender offers, rights is-

sues, bonus issues, spin-offs), index fixings, option exercises, defaults, swap resets, novations, cash-settlement determinations, IBOR or other index cessation fallbacks, and inbound confirmations from counterparties or platforms. Fields: `kind` (`dividend`, `split`, `merger`, `tender_offer`, `rights_issue`, `bonus_issue`, `spin_off`, `fixing`, `exercise`, `default`, `reset`, `cash_settlement_amount_determined`, `index_cessation_fallback`, `confirmation_received`), `instrument_ref` or `trade_ref` (one is required), `effective_date`, plus kind-specific fields: `ex_date`, `record_date`, `payment_date`, `cash_amount`, `currency` for dividends; `ratio` for splits; `settlement_amount`, `settlement_currency` for `cash_settlement_amount_determined` (the final cash amount of a cash-settled option at expiry or the cash leg of a physically-settled corporate action); `replaced_index`, `replacement_index`, `spread_adjustment` for `index_cessation_fallback` (recorded when an IBOR or other index ceases and a fallback index plus spread takes its place on referencing contracts); `external_message_id` for inbound confirmations. Carrier: issuers, exchanges, calculation agents, and confirmation platforms (FpML feeds, ISDA Create, DRR submission acknowledgements). Securities-lending events (locate, recall, manufactured payment, rehypothecation) are out of scope for the MVP. The `kind` list above is closed for the MVP; admitting a new kind requires a catalogue revision, the same discipline applied to the observation status enum.

The list mixes events at two layers: most kinds (`dividend`, `split`, `merger`, `fixing`, `exercise`, `default`, `reset`, and the new corporate-action and settlement kinds) are business-event qualifiers — they describe what happened to the instrument or trade in the market. `confirmation_received` is at a different layer: it is an operational-message event describing a wire arrival at our boundary, and it does not by itself change the economic state of any contract. Both layers belong in the same table because both feed downstream reconciliation, but a consumer that wants only business events filters on `kind != confirmation_received`.

We need lifecycle events on the same provenance and bitemporal footing as everything else because a misrecorded dividend or a missed corporate action shifts every position downstream of it, and the cash impact has to be reproducible weeks later.

CDM mapping. CDM splits the lifecycle across two layers, which is why our `kind` list also crosses two layers. `BusinessEvent` (in `event-common-type.rosetta`) extends `EventInstruction` (in `event-workflow-type.rosetta`), which carries `instruction : Instruction (0..*)` and `corporateActionIntent : CorporateActionTypeEnum (0..1)`. Each `Instruction` references a `PrimitiveInstruction` (in `event-common-type.rosetta`), which is a record with optional sub-instruction fields — not a choice — namely `contractFormation`, `execution`, `exercise`, `partyChange`, `quantityChange`, `reset`, `split`, `termsChange`, `transfer`, `indexTransition`, `stockSplit`, `observation`, `valuation`; these are fields, not choice variants, and any combination can be set on a single primitive. `Reset` is its own type (same file), which joins product definition to `Observation`. Corporate actions are tagged via `CorporateActionTypeEnum` in `event-common-enum.rosetta`, which enumerates `CashDividend`, `StockDividend`, `StockSplit`, `ReverseStockSplit`, `Merger`, `Takeover`, `RightsIssue`, `BonusIssue`, `SpinOff`, `Delisting`, `Liquidation`, `EarlyRedemption`, and others. Operational-message events (our `confirmation_received`) live in the FpML-ingest layer, in the `ingest-fpml-confirmation-*-func.rosetta` family (e.g. `ingest-fpml-confirmation-message-func.rosetta`, `ingest-fpml-confirmation-workflowstep-func.rosetta`) — these are inbound-message handlers, not business events. `cash_settlement_amount_determined` maps to the `transfer` field on `PrimitiveInstruction` (which carries a `TransferInstruction` that resolves to a `Transfer` choice over `TransferBase extends AssetFlowBase`) or to a `Reset` depending on whether the determination produces a settlement transfer or an interim observation; `index_cessation_fallback` maps to `IndexTransitionInstruction`. The Ledger's flat `LifecycleEvent` table is the storage shape; on ingest, CDM business-events are flattened into a row with their `CorporateActionTypeEnum` or instruction-field name mapped to our local `kind` value.

Example. An Apple dividend: `(kind=dividend, instrument_ref=AAPL, effective_date=2026-05-08, ex_date=2026-05-08, record_date=2026-05-12,`

payment_date=2026-05-15, cash_amount=0.25, currency=USD).

2.12 Legal agreements

A legal agreement is a bilateral document stack governing how two parties trade. The stack has four kinds in order: a Master agreement, a Schedule that amends it, a Credit Support Annex (CSA) for collateral terms, and a Paragraph 11 election that sets specific CSA parameters. Fields: `agreement_id`, `agreement_type` (`Master`, `Schedule`, `CSA`, `Para11`), `parties` (exactly two LEIs for bilateral agreements), `governing_law`, `signed_date`, `effective_date`, `document_hash` (content hash for retrieval), `parent_agreement_id` (a Schedule references its Master, and so on up the stack). Carrier: contract management system; counterparties on signature.

The CDM defines the named containers for these agreement types (the `Master`/`Schedule`/`CSA`/`Para11` field structures and the LEI-and-governing-law fields on each); the override-walk policy below is Ledger-local. To resolve an attribute (governing law, eligible collateral, threshold amount, minimum-transfer amount) for a given trade:

- Walk the stack in order: Master, then Schedule, then CSA, then Paragraph 11.
- At each level, if the attribute is set, it overrides whatever lower-precedence level set it.
- "Set" includes an explicit `null` value: a child agreement that explicitly nulls a parent's value (e.g. a Paragraph 11 setting the threshold from $10mn back to zero, or a CSA explicitly clearing an eligible-collateral entry) overrides the parent. Only a missing field — the field is not present on this level's document at all — means inherit from the level above.
- The CSA carries its own `governing_law`: a 2002 ISDA Master under English law sitting under a CSA under New York law is a routine arrangement, not an exception, and the walk treats CSA `governing_law` as a normal override node.
- The walk's result for a given attribute is what the platform uses for valuation, collateral, and dispute purposes.

Stored agreement documents are verified against `document_hash` on retrieval: a stored bytestream whose hash does not match the recorded `document_hash` is rejected, the retrieval is failed, and the mismatch is logged as an exception event for operations review.

We need the explicit override walk because two implementers comparing notes on the same trade reconstruct different governing terms when the walk is left implicit, and a dispute is exactly when that ambiguity is most expensive.

CDM mapping. The agreement-record base type is `LegalAgreement extends LegalAgreementBase` (`legaldocumentation-common-type.rosetta`), with `LegalAgreementIdentification` carrying `agreementName : AgreementName`, `publisher : LegalAgreementPublisherEnum (0..1)`, `governingLaw : GoverningLawEnum (0..1)`, and `vintage : int (0..1)` (the year, e.g. 2013 for the ISDA 2013 CSA). The agreement-type discriminator is `LegalAgreementTypeEnum` in `legaldocumentation-common-enum.rosetta`, whose values are `MasterAgreement`, `CreditSupportAgreement`, `Confirmation`, `MasterConfirmation`, `BrokerConfirmation`, `SecurityAgreement`, `Other`. CDM does NOT separate "Schedule" from Master or "Paragraph 11" from CSA as enum values — a Schedule is part of the `MasterAgreement` elections (`legaldocumentation-master-isda-type.rosetta`, `type MasterAgreement extends MasterAgreementBase`) and Paragraph 11 is part of the CSA elections (`legaldocumentation-csa-type.rosetta`, with the `CreditSupportAgreementInitialMargin-Elections`, `VariationMarginElections`, and `LegacyElections` branches). The Ledger's 4-tier flat stack is a local denormalisation: we store each tier as its own row so the override walk reads as a list of rows joined by `parent_agreement_id` rather than a deeply nested CDM document. Document hash and the override-walk policy itself are absent from CDM and remain Ledger-local. Eligible collateral rides on `CollateralProvisions` (in `product-collateral-type.rosetta`, carrying `collateralType`, `eligibleCollateral`, `substitutionProvisions`); thresholds and minimum-transfer amounts ride on `CreditSupportObligationsBase` and its specialisations `CreditSupportObligationsInitialMargin`, `VariationMargin`, `Legacy` (in

`legaldocumentation-csa-type.rosetta`), reachable from each CSA-elections type via its `creditSupportObligations` attribute.

Example. A 2002 ISDA Master `(MA-001)` between Bank A (LEI `ABC...`) and Hedge Fund B (LEI `DEF...`), English law, signed 2025-01-15. Schedule `SC-001` references `MA-001`. CSA `CSA-001` references `SC-001`, sets `governing_law=New York` for the collateral arrangement, and restricts eligible collateral to USD cash. Paragraph 11 `P11-001` references `CSA-001` and sets the threshold to $10mn.

2.13 Reference tables

A small mixed bag of authoritative cross-cutting tables, each with its own field shape.

- *Sanctions list*: `list_authority` (`OFAC`, `UN`, `EU`), `entity_name`, `lei` (where known), `effective_date`, `delisting_date` (nullable; the date the entity was removed from the list, recorded as a new row rather than by mutating the listing row).
- *Withholding-tax*: `payer_jurisdiction`, `payee_jurisdiction`, `instrument_kind`, `rate`, `treaty_ref`.
- *Index composition*: `index_name`, `effective_date`, `members` (list of `(ticker, weight)`).

We need these tables on the same provenance and bitemporal footing as the rest of the catalogue because their values determine cashflows (withholding rates, index weightings) and trading eligibility (sanctions); a quietly-updated sanctions list with no record of the prior version cannot answer the question "did we trade with a sanctioned party last March?".

CDM mapping. Mostly missing. Sanctions lists are not modelled in CDM; the Ledger carries them as a local table joined to CDM `Party` by LEI. Withholding-tax schedules have no CDM type at the table level either (CDM models the per-cashflow tax withheld inside payouts, but not the rate-by-jurisdiction master). Index composition is the one item with some CDM presence: a basket index can be expressed as `Basket extends AssetBase` (`observable-asset-type.rosetta`) with `BasketConstituent extends Observable` entries, each carrying a weight via `ConstituentWeight` (`product-template-type.rosetta`); for a named market index like S&P 500 the composition is normally implicit in the `EquityIndex` reference and the Ledger keeps the concrete weighted constituents in its own table. An extension would add `SanctionsList` and `WithholdingTaxSchedule` root types alongside the existing reference-data types.

Example (index composition). S&P 500 as of 2026-04-30: `(SPX, 2026-04-30, [(AAPL, 0.0720), (MSFT, 0.0670), (NVDA, 0.0530), ...])`.

3 Provenance and traceability

Every record in every concept above carries four provenance fields, in addition to whatever fields the concept itself defines.

- `source`: a named string identifying who said this. For a quote that might be `"Bloomberg BPIPE"`; for an LEI record, `"GLEIF"`; for a calibrated curve, `"internal/calibration-svc/v2.3.1"`. The permitted set of `source` strings is maintained in a small source-registry reference table; ingest rejects records whose `source` is not in the registry, so the field stays a controlled vocabulary rather than a free-text column.
- `t_obs` (observation time): the wall-clock instant the underlying event occurred in the world — the moment the price printed, the moment a corporate-action effective date applied, the moment a curve was calibrated. Stored in UTC.
- `t_known` (known time): the wall-clock instant our system first received and admitted the record. Stored in UTC.
- `restates_ref`: a nullable foreign key (by surrogate id) pointing into the same table at the prior row this record corrects. `null` on an original observation; populated only on a restatement. The mechanics of restatement are detailed in Section 4; the field is named here because, like the other

three, it is carried on every record in every concept that admits restatement and is therefore part of provenance, not part of any concept's own field list.

We need `source` on every record because reconciliation, dispute resolution, and vendor-quality monitoring all begin with the question "where did this number come from?", and answering that with an `ETL_LOAD_DT` timestamp instead of a named upstream is the difference between a five-minute investigation and a five-day one. We need the two timestamps because the system is not the world: we learn things later than they happen, sometimes much later, and we need to be able to distinguish "this happened then" from "we found out about it then".

The constraint $t_{\text{obs}} \leq t_{\text{known}}$ holds on every record. A record claiming knowledge of an event before it happened is rejected at ingest; that is a clock or feed bug, not a legitimate observation. A small tolerance (on the order of one second) is permitted at ingest to absorb routine clock skew between our system and vendor feeds — a vendor whose clock runs 200ms ahead of ours would otherwise produce records that fail the constraint legitimately. Records exceeding the tolerance are quarantined and flagged for operations review, not silently accepted.

Example. A trade print on Apple at 16:00 ET on 2026-04-30, received by our system at 16:00:01 ET, then mirrored as: `source="NASDAQ TotalView"`, `t_obs=2026-04-30 20:00:00Z`, `t_known=2026-04-30 20:00:01Z`.

Out of scope: cryptographic signatures. The Ledger platform is internally operated and is not exposed to adversarial counterparties at the ingest boundary; the named `source` field is the trust signal at this stage. Cryptographic signing of inbound payloads, an attestor key registry, and signature verification are noted as future hardening for a later document.

4 Bitemporal storage

The two timestamps on every record support two query modes. The reader who has just learned the names `t_obs` and `t_known` can think about it this way: `t_obs` is when the world did something; `t_known` is when we found out about it. They are usually close together. Sometimes they are not.

No record is overwritten. When a vendor or a counterparty corrects a record for an earlier t_{obs}, the correction is stored as a *new row* with the same t_{obs}, a later t_{known}, and a pointer back to the row it restates. The original row is not modified. We need this rule because the question "what did we believe yesterday?" has a single correct answer only if yesterday's belief is still on disk; an in-place update destroys that answer permanently.

Two query modes, both supported.

- *What did we know at time T?* The query returns the snapshot of the table as it was known at T: every row whose $t_{\text{known}} \leq T$, with the latest such row per business key winning. This is the question an auditor asks: "reproduce the NAV you published on 28 April with the data you had on 28 April."
- *Given everything we know now, what is our best estimate of the truth at time T_o?* The query returns the row with the latest t_{known} among all rows whose $t_{\text{obs}} = T_o$. This is the question a risk team asks: "with corrections applied, what is our best read on yesterday's exposure?"

Neither query mode is derivable from the other. A single "as-of" timestamp on each row cannot answer both.

Worked example: a vendor restates EUR/USD. On 2026-04-27 at 15:30 UTC the vendor publishes EUR/USD = 1.0852 with t_{obs} = 2026-04-27 15:30:00Z and t_{known} = 2026-04-27 15:30:00Z (received the same instant). On 2026-04-30 at 09:00 UTC the vendor restates the same observation to 1.0853. The table now holds two rows:

Row	value	t_{obs}	t_{known}	restates
R_1	1.0852	2026-04-27 15:30:00Z	2026-04-27 15:30:00Z	—
R_2	1.0853	2026-04-27 15:30:00Z	2026-04-30 09:00:00Z	R_1

Query 1: what did we know at 2026-04-29 00:00:00Z? Answer: 1.0852. As of that knowledge time R_2 did not yet exist; the only row whose $t_{\text{known}} \leq$ 2026-04-29 was R_1. This is the answer the auditor needs to reproduce a NAV published on 28 April.

Query 2: with everything we know on 2026-04-30 at 10:00 UTC, what was the truth at 2026-04-27 15:30Z? Answer: 1.0853. Among rows with matching t_{obs}, the row with the latest $t_{\text{known}} \leq$ 2026-04-30 10:00 is R_2. This is the answer the risk team needs when restating yesterday's exposure with current best knowledge.

This two-axis discipline applies to every concept in the catalogue that admits restatement: market observations, calibrated objects, lifecycle events, party records, instrument masters, legal agreements, and reference tables. A narrow carve-out is permitted only for tables whose values have never influenced a published number: a pure-internal lookup that nothing downstream reads, or a feature-flag table the platform itself owns. Any table that feeds admission, calibration, valuation, reporting, or any consumed snapshot must be bitemporal — no carve-out, no exception. The no-overwrite rule is a constraint on the storage technology as well as on the discipline: a backing store that only supports update-in-place semantics cannot meet this audit floor, and the warehouse design must therefore use an append-only or row-versioned store for the bitemporal tables.

5 Loading for the pricing library

The catalogue defines what we store. This section defines the storage shape the pricing library needs in order to load that data quickly. The pricing library is the component that produces a price for an instrument or a trade. To produce one price it fetches the instrument's static record, the observable definitions it depends on, the latest raw observations (when the price is a re-mark rather than a calibrated one), the calibrated market objects that feed valuation, and any lifecycle events that have changed the trade's state. The round-trip time for producing a single price is dominated by how fast it can fetch this data: when fetching is slow, every price is slow. A pricing batch that revalues a book at end-of-day issues many thousands of these reads; a real-time risk recalculation issues them on each tick. The catalogue's storage shape must therefore support a small set of access patterns as single indexed lookups or bounded range scans, not as scans of restatement history.

The concrete access patterns the storage must support efficiently are these.

- *Curve or surface by key.* "Give me the as-of-known-time snapshot of curve K" where K is `(kind, currency_or_underlier, valuation_date)` must be a single indexed lookup. The natural index is `(kind, currency_or_underlier, valuation_date, t_known)` with a latest-row-per-key resolver. Without this index, every price re-mark walks the calibrated-objects table.
- *All curves for a valuation date.* "Give me every calibrated object whose `valuation_date = D`" must be index-supported on `valuation_date`, because the end-of-day pricing batch enumerates calibrated objects this way before issuing per-curve fetches.
- *Instrument by identifier and known time.* "Give me the instrument master row for `(authority, instrument_id)` at known time T" must be a single lookup. Authority here is the identifier source (ISIN, FIGI, CUSIP, internal id, LEI for parties). The natural index is `(authority, instrument_id, t_known)`.
- *Latest observation for a definition.* "Give me the latest as-of-now value for observable definition O" must be a single lookup keyed by `definition_ref`, returning the row with the maximum t_{known} whose t_{obs} falls in the requested window. The raw-observations table is the biggest table in the catalogue, so this query without an index is the single largest pricing-batch hazard.
- *Lifecycle events on an instrument over a window.* "Give me all lifecycle events for instrument I with `effective_date` in $[A, B]$" must be a bounded range scan, indexed on `(instrument_ref, effective_date)`. Pricing a bond mid-life needs every coupon, every fixing, every call event in the period; pricing an equity needs dividends and splits.
- *Override-walk for legal agreements.* Given a trade, fetching the four agreement rows that make up the trade's stack (Master, Schedule, CSA, Paragraph 11) must traverse `parent_agreement_id`

in at most four indexed reads, not by scanning all agreements between the two parties.

We need these access patterns named explicitly because bitemporal storage's flexibility is bought at the cost of larger tables and longer history; without indexes on the natural access keys, every pricing read scales with the total history of restatements rather than with the size of the current snapshot, and that cost is felt on every price.

Calibrated curves and surfaces are stored as the knot-point representation named in the catalogue, deserialisable directly into the pricing library's curve and surface objects without re-parsing a vendor-side format. The same applies to instrument masters: the bond record's day-count, frequency, and schedule fields are flat scalars, not a CDM payout structure to be resolved at query time. The pricing library should never have to traverse a CDM `NonTransferableProduct` tree to recover a coupon rate that the catalogue already holds as a column.

A separate "current" projection — the latest non-restated row per business key, materialised into a denormalised hot-path table — may be maintained for the highest-volume reads (latest observations, current calibrated curves, current instrument masters). The projection is a cache and stays consistent with the bitemporal source by re-deriving on every write; the bitemporal table remains the system of record. A query that asks "what did we know at time T" for a T in the past must read the bitemporal table directly. The projection exists only so that "what is true now" does not pay the bitemporal-query cost on every read.

Order-of-magnitude sizing as design context, not as an operating-envelope number: the platform expects to hold tens of thousands of calibrated curves and surfaces per valuation date, millions of raw observations per business day, hundreds of thousands of instrument master rows, and a low six-figure count of active lifecycle events at any time. Specific latency and throughput targets are not set here; the data engineering team picks indexes, partitioning, and storage technology to meet them and reports back.

6 CDM as the wire vocabulary

CDM is the FINOS Common Domain Model, expressed in the Rosetta DSL and distributed as `*.rosetta` source files under `rosetta-source/src/main/rosetta/`. It defines named types for products, parties, events, observables, and legal documentation. CDM is the direction every counterparty, vendor, and regulator we exchange data with is converging on. We adopt it because private schemas multiply reconciliation cost: every pair of systems that disagree on field names or field meanings creates a translation layer, and translation layers are where data quietly goes wrong.

The rule is simple. Where CDM defines a type for a concept in the catalogue, the CDM type is the wire format on ingest and the storage shape in the warehouse; the per-concept CDM mapping paragraphs in Section 2 name the exact Rosetta type and source file for each concept. Where CDM does not yet define a type — calibrated curves and surfaces, observable definitions as a separate "key" from values, sanctions lists, withholding-tax schedules, holiday-date resolution tables, sector classifications — the storage is a local shape and the gap is recorded in the data dictionary. The list of gaps is finite and stable enough for an MVP; an extension can be drafted later in the same Rosetta DSL, alongside the existing CDM source files, without changing this document's catalogue. The Ledger version that this document targets is CDM 6.0.0 (master branch, verified at authoring time); future CDM releases that fill these gaps will simplify the mapping but should not invalidate the catalogue.

A clarifier on what this rule does and does not pin: CDM defines the *field structure* — the names, the types, the nesting, the cardinalities — of each record. It does not pin the *wire encoding bytes*: an FpML-style XML payload, an Avro container, a Protobuf message, or a JSON document can each carry the same CDM-shaped record. The FpML ingest functions in the `ingest-fpml-confirmation-*-func.rosetta` family illustrate the layering: the functions are CDM-typed but their inputs are FpML XML and their outputs are CDM `WorkflowStep` or `TradeState` records. Storage technology and wire encoding choices (listed as out of scope in Sec-

tion 7) are therefore independent of the CDM rule above; CDM constrains the schema, not the serialisation.

7 What this document does not cover

The following are deliberately out of scope for this document.

- *The move stream, position state, and unit balances.* These are the platform's internal record of what happened, derived from ingested data; they are written by the platform, not by the data team.
- *Valuation, pricing methodology, and Greeks.* The data team supplies calibrated market objects; the mathematics that consumes them is the valuation team's responsibility.
- *Cryptographic signing of inbound records, attestor key registry, key rotation.* Future hardening, not MVP.
- *Securities-lending lifecycle events* (locate, recall, manufactured payment, rehypothecation). A separate workstream; the lifecycle concept above is sized for the non-SBL events.
- *CCP initial-margin schedules.* The platform consumes margin calls (events), not schedules.
- *Storage technology and wire encoding choices.* This document specifies data shape; whether the warehouse is columnar Parquet, an immutable log, or a relational store, and whether the wire format is Avro, Protobuf, JSON, or CBOR, are separate decisions. CDM pins the field structure regardless.
- *Vendor selection, KYC/AML controls beyond the fields they require, legal-entity onboarding workflows.*

8 Glossary

Bitemporal
Two-axis indexing of records by observation time and known time; both axes are queryable.

CDM
FINOS Common Domain Model, the shared field-structure vocabulary used as the wire format and storage shape wherever it defines a type that fits.

Calibrated market object
A curve or surface (yield curve, vol surface, FX surface, hazard curve) fitted from raw observations and stored as a list of knot points with a reference back to the inputs that produced it.

GLEIF
Global Legal Entity Identifier Foundation, the authority that issues LEIs.

Known time (t_{known})
The wall-clock instant our system first received and admitted a record.

LEI
Legal Entity Identifier, ISO 17442; a 20-character identifier for a legal entity.

Lifecycle event
An event that happens to an instrument or a trade after it begins to exist: dividend, split, merger, fixing, exercise, default, reset, inbound confirmation.

MIC
Market Identifier Code, ISO 10383; a four-character identifier for a trading venue.

Observable definition
The key under which raw market values are recorded; identifies what is being observed without carrying any value.

Observation time (t_{obs})
The wall-clock instant the underlying event occurred in the world.

OTC product
A contract whose terms are negotiated bilaterally rather than listed on an exchange; the substrate stores the terms, not a vendor identifier.

Restatement
A correction to a prior record, stored as a new row with the same t_{obs}, a later t_{known}, and a pointer back to the row it restates.

`restates_ref`
The nullable foreign key on a record pointing into the same table at the prior row this record restates; `null` on an original, populated on a correction.

Source
A named string identifying who delivered a record (vendor name, authority name, internal service name). The permitted set is held in a source-registry reference table.

`value_time`
On raw market observations, the source-reported instant of the underlying market event (the printed quote or trade time on the vendor wire). Distinct from t_{obs} in that t_{obs} is the same instant normalised to UTC; they usually coincide, and diverge when the source uses a non-UTC clock or restates a printed time.

Unit
Any traded thing recorded as a balance in a wallet: a currency, a share, a bond, an OTC contract, an on-chain obligation.

Wallet
A named account inside the Ledger holding quantities of units. Not a custody account or a bank account.

Document Version: v2.0 **Date:** May 2026 **Classification:** MVP specification, data engineering audience.